An Analysis of Federated learning

Federated learning

Federated learning -- Part 1

Technical Federated learning requires frequent communication between nodes during the learning process. Thus, it requires not only enough local computing power and memory, but also high bandwidth connections to be able to exchange parameters of the machine learning model. However, the technology also avoids data communication, which can require significant resources before starting centralized machine learning. Nevertheless, the devices typically employed in federated learning are communication-constrained, for example IoT devices or smartphones are generally connected to Wi-Fi networks, thus, even if the models are commonly less expensive to be transmitted compared to raw data, federated learning mechanisms may not be suitable in their general form. Federated learning raises several statistical challenges:

Federated learning -- Part 2

Current research topics Federated learning has started to emerge as an important research topic in 2015 and 2016, with the first publications on federated averaging in telecommunication settings. Before that, in a thesis work titled "A Framework for Multi-source Prefetching Through Adaptive Weight", an approach to aggregate predictions from multiple models trained at three location of a request response cycle with was proposed. Another important aspect of active research is the reduction of the communication burden during the federated learning process. In 2017 and 2018, publications have emphasized the development of resource allocation strategies, especially to reduce communication requirements between nodes with gossip algorithms as well as on the characterization of the robustness to differential privacy attacks. Other research activities focus on the reduction of the bandwidth during training through sparsification and quantization methods, where the machine learning models are sparsified and/or compressed before they are shared with other nodes. Developing ultra-light DNN architectures is essential for device-/edge- learning and recent work recognises both the energy efficiency requirements for future federated learning and the need to compress deep learning, especially during learning. Recent research advancements are starting to consider real-world propagating channels as in previous implementations ideal channels were assumed. Another active direction of research is to develop Federated learning for training heterogeneous local models with varying computation complexities and producing a single powerful global inference model. A learning framework named Assisted learning was recently developed to improve each agent's learning capabilities without transmitting private data, models, and even learning objectives. Compared with Federated learning that often requires a central controller to orchestrate the learning and optimization, Assisted learning aims to provide protocols for the agents to optimize and learn among themselves without a global model.

Federated learning -- Part 3

Federated stochastic gradient descent (FedSGD) Stochastic gradient descent is an approach used in deep learning, where gradients are computed on a random subset of the total dataset and then used to make one step of the gradient descent.. Federated stochastic gradient descent is the analog of this algorithm to the federated setting, but uses a random subset of the nodes, each node using all its data. The server averages the gradients in proportion to the number of training data on each node, and uses the average to make a gradient descent step.

Federated learning -- Part 4

Optimizing the objective function f (x 1 , … , x K) {\displaystyle f(\mathbf {x} _{1},\dots ,\mathbf {x} _{K})} . Achieving consensus on x i {\displaystyle \mathbf {x} _{i}} . In other words, x 1 , … , x K {\displaystyle \mathbf {x} _{1},\dots ,\mathbf {x} _{K}} converge to some common x {\displaystyle \mathbf {x} } at the end of the training process.

Federated learning -- Part 5

Heterogeneous federated learning Most of the existing federated learning strategies assume that local models share the same global model architecture. Recently, a new federated learning framework named HeteroFL was developed to address heterogeneous clients equipped with very different computation and communication capabilities. The HeteroFL technique can enable the training of heterogeneous local models with dynamically varying computation and non-IID data complexities while still producing a single accurate global inference model.

Federated learning -- Part 6

Transportation: self-driving cars Self-driving cars encapsulate many machine learning technologies to function: computer vision for analyzing obstacles, machine learning for adapting their pace to the environment (e.g., bumpiness of the road). Due to the potential high number of self-driving cars and the need for them to quickly respond to real world situations, traditional cloud approach may generate safety risks. Federated learning can represent a solution for limiting volume of data transfer and accelerating learning processes.

Federated learning -- Part 7

Iterative learning The iterative process of federated learning is composed of a series of fundamental client-server interactions, each of which is known as a federated learning round. Each round of this process consists in transmitting the current global model state to participating nodes, training local models on these local nodes to produce a set of potential model updates at each node, and then aggregating and processing these local updates into a single global update and applying it to the global model. In the methodology below, a central server is used for aggregation, while local nodes perform local training depending on the central server's orders. However, other strategies lead to the same results without central servers, in a peer-to-peer approach, using gossip or consensus methodologies. Assuming a federated round composed by one iteration of the learning process, the learning procedure can be summarized as follows:

Federated learning -- Part 8

HyFDCA is a provably convergent primal-dual algorithm for hybrid FL in at least the following settings. Hybrid Federated Setting with Complete Client Participation Horizontal Federated Setting with Random Subsets of Available Clients The authors show HyFDCA enjoys a convergence rate of O(1⁄t) which matches the convergence rate of FedAvg (see below). Vertical Federated Setting with Incomplete Client Participation The authors show HyFDCA enjoys a convergence rate of O(log(t)⁄t) whereas FedBCD exhibits a slower O(1⁄sqrt(t)) convergence rate and requires full client participation. HyFDCA provides the privacy steps that ensure privacy of client data in the primal-dual setting. These principles apply to future efforts in developing primal-dual algorithms for FL. HyFDCA empirically outperforms HyFEM and FedAvg in loss function value and validation accuracy across a multitude of problem settings and datasets (see below for more details). The authors also introduce a hyperparameter selection framework for FL with competing metrics using ideas from multiobjective optimization. There is only one other algorithm that focuses on hybrid FL, HyFEM proposed by Zhang et al. (2020). This algorithm uses a feature matching formulation that balances clients building accurate local models and the server learning an accurate global model. This requires a matching regularizer constant that must be tuned based on user goals and results in disparate local and global models. Furthermore, the convergence results provided for HyFEM only prove convergence of the matching formulation not of the original global problem. This work is substantially different than HyFDCA's approach which uses data on local clients to build a global model that converges to the same solution as if the model was trained centrally. Furthermore, the local and global models are synchronized and do not require the adjustment of a matching parameter between local and global models. However, HyFEM is suitable for a vast array of architectures including deep learning architectures, whereas HyFDCA is designed for convex problems like logistic regression and support vector machines. HyFDCA is empirically benchmarked against the aforementioned HyFEM as well as the popular FedAvg in solving convex problem (specifically classification problems) for several popular datasets (MNIST, Covtype, and News20). The authors found HyFDCA converges to a lower loss value and higher validation accuracy in less overall time in 33 of 36 comparisons examined and 36 of 36 comparisons examined with respect to the number of outer iterations. Lastly, HyFDCA only requires tuning of one hyperparameter, the number of inner iterations, as opposed to FedAvg (which requires tuning three) or HyFEM (which requires tuning four). In addition to FedAvg and HyFEM being quite difficult to optimize hyperparameters in turn greatly affecting convergence, HyFDCA's single hyperparameter allows for simpler practical implementations and hyperparameter selection methodologies.